

Actividad 2 — El clásico Nibbles (la viborita)

Duración: 50 min · **Modalidad:** individual · **Entrega:** repo GitHub público

1. Objetivos

- Modelar el juego sobre una **cuadrícula** (grid).
- Usar **listas** para representar el cuerpo de la serpiente.
- Detectar colisiones (con la manzana, con el borde y consigo misma).
- Implementar el sistema de puntaje.

2. Marco teórico

Nibbles es un juego arcade clásico (nació en los años 80). Una serpiente avanza continuamente; cuando come una manzana **crece** y suma puntos; si choca contra la pared o contra su propio cuerpo, muere.

Truco clave: no movemos “cada parte” de la serpiente. Guardamos el cuerpo como una **lista de coordenadas** donde la cabeza es la posición 0. En cada cuadro:

1. Se **inserta** la nueva cabeza (`insert(0, ...)`).
2. Si no comió, se **elimina** la cola (`pop()`).

Así la serpiente avanza y crece sin escribir física compleja.

La pantalla se divide en **celdas** de 30x30 px. Una coordenada (columna, fila) se convierte en píxeles multiplicando por el tamaño de celda.

3. Paso a paso

Creá la carpeta y el archivo:

```
cd mis-videojuegos
mkdir clase_02_nibbles
```

Escribí `clase_02_nibbles/nibbles.py`:

```
import random
import sys
import pygame
```

```
pygame.init()
```

```
ANCHO, ALTO = 600, 600
CELDA = 30
COLUMNAS = ANCHO // CELDA
FILAS = ALTO // CELDA
```

```

pantalla = pygame.display.set_mode((ANCHO, ALTO))
reloj = pygame.time.Clock()

# El cuerpo: lista de (columna, fila). La cabeza es el primer elemento.
serpiente = [(5, 5)]
direccion = (1, 0) # (dx, dy) -> derecha

def manzana_nueva():
    while True:
        m = (random.randint(0, COLUMNAS - 1), random.randint(0, FILAS -
1))
        if m not in serpiente:
            return m

manzana = manzana_nueva()
puntos = 0

def dibujar_celda(pos, color):
    pygame.draw.rect(pantalla, color,
                     (pos[0] * CELDA, pos[1] * CELDA, CELDA - 2, CELDA -
2))

ejecutando = True
while ejecutando:
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            ejecutando = False
        elif evento.type == pygame.KEYDOWN:
            # No dejar que gire 180 grados (no puede ir para atrás)
            if evento.key == pygame.K_UP and direccion != (0, 1):
                direccion = (0, -1)
            elif evento.key == pygame.K_DOWN and direccion != (0, -1):
                direccion = (0, 1)
            elif evento.key == pygame.K_LEFT and direccion != (1, 0):
                direccion = (-1, 0)
            elif evento.key == pygame.K_RIGHT and direccion != (-1, 0):
                direccion = (1, 0)

    # 1) Nueva cabeza
    cabeza = (serpiente[0][0] + direccion[0], serpiente[0][1] + direccion
[1])
    serpiente.insert(0, cabeza)

    # 2) ¿Comió?
    if cabeza == manzana:
        puntos += 1
        manzana = manzana_nueva()
    else:

```

```

        serpiente.pop() # no comió -> se achica por el final

# 3) ¿Chocó con el borde o consigo misma?
if (cabeza[0] < 0 or cabeza[0] >= COLUMNAS or
    cabeza[1] < 0 or cabeza[1] >= FILAS or
    cabeza in serpiente[1:]):
    ejecutando = False

# 4) Dibujar
pantalla.fill((10, 10, 15))
for segmento in serpiente:
    dibujar_celda(segmento, (0, 220, 60))
dibujar_celda(manzana, (230, 40, 40))
pygame.display.set_caption(f"Puntos: {puntos}")
pygame.display.flip()
reloj.tick(10) # 10 FPS: velocidad clásica

pygame.quit()
print(f"Fin del juego. Puntos: {puntos}")
sys.exit()

```

4. Probalo y entendolo

python clase_02_nibbles/nibbles.py

Respondete antes de seguir:

- ¿Por qué la serpiente avanza sola sin apretar nada?
- ¿Qué pasa si cambiás reloj.tick(10) por tick(20)?
- ¿Por qué cabeza in serpiente[1:] detecta la mordida a sí misma? (fijate qué significa el [1:]).

5. Actividad de cierre (para el commit)

1. Agregá una **manzana dorada** que vale 5 puntos y aparece con menos frecuencia.
2. Mostrá el **récord** guardándolo en un archivo récord.txt:

```

try:
    with open("record.txt") as f:
        record = int(f.read())
except FileNotFoundError:
    record = 0
if puntos > record:
    record = puntos
    with open("record.txt", "w") as f:
        f.write(str(record))

```

3. Hacé commit y push:

```
git add .
git commit -m "Clase 02: Nibbles completo con récord"
git push
```

6. Usando IA en esta clase (prompts sugeridos)

Prompt 1: “¿Por qué en el juego Nibbles es mejor guardar la serpiente como una lista de coordenadas en vez de mover cada segmento por separado?”

Prompt 2: “Agregá a mi código de Nibbles una manzana dorada que da 5 puntos. Mi código: [pegá el código].”

Prompt 3: “Mi serpiente atraviesa la pared. Este es mi código: [pegá el código]. ¿Cómo hago que el juego termine cuando toca el borde?”

Consejo: no pegues código de internet sin revisarlo. Compará siempre con lo que viste en clase.

7. Rúbrica de evaluación

Criterio	Logrado	En proceso	No logrado
Movimiento y crecimiento	Come y crece	Come pero no crece	No funciona
Colisiones	Pared y cuerpo	Solo pared	Ninguna
Puntaje y récord	Ambos	Solo puntaje	Ninguno
Git	2 commits	1 commit	Sin commits